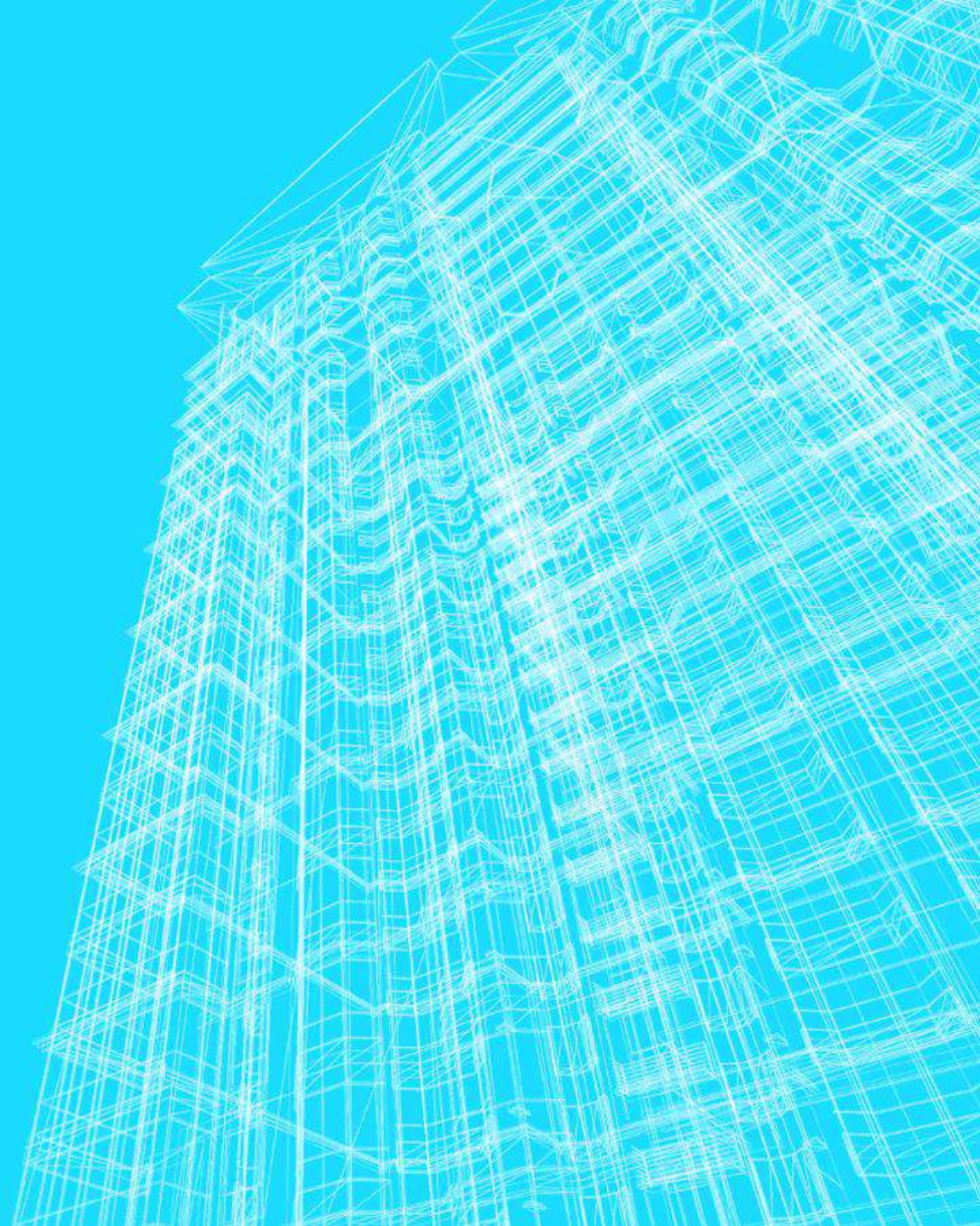
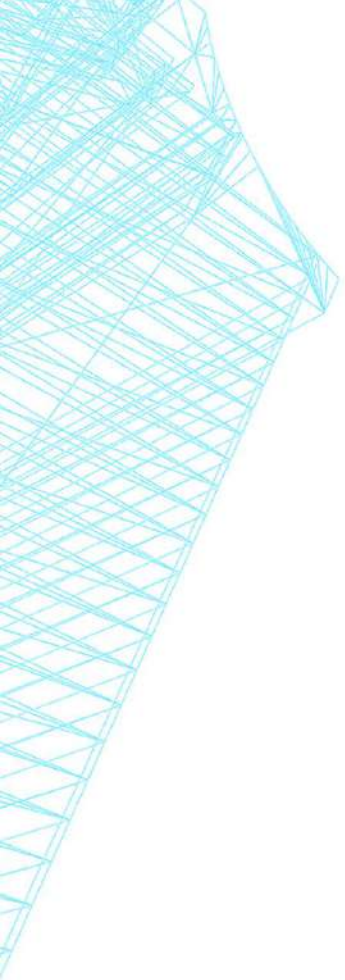


NOTAÇÕES O , Ω E Θ . COMPORTAMENTO ASSINTÓTICO E CLASSES DE COMPORTAMENTO.

João Pedro Oliveira Batisteli





CUSTO DE EXECUÇÃO DE ALGORITMOS

- Utilizamos uma função matemática $f(n)$ para representar o custo estimado de um determinado algoritmo.
- Valores pequenos de n : mesmo os algoritmos ruins custam pouco.
- Logo, a análise de algoritmos é realizada para valores grandes de n .

CUSTO DE EXECUÇÃO DE ALGORITMOS

- Algoritmo com $f(n) = 2^n$.
- Computador que realize 10^9 (1 bilhão) de instruções por segundo.

n	f(n)	Tempo estimado
5	$2^5 = 32$	0,000000032 seg
10	$2^{10} = 1.024$	0,000001024 seg
20	$2^{20} = 1.048.576$	0,001048576 seg
40	$2^{40} = 1.099.511.627.776$	1.099 seg = 18 min
80	$2^{80} = 1.208.925.819.614.629.174.706.176$	$1,2 \times 10^{21}$ seg = 38.308.547 anos

COMPORTAMENTO ASSINTÓTICO

- O comportamento assintótico de $f(n)$ representa o limite do comportamento do custo quando n cresce.
 - Crescimento assintótico da operação mais relevante.
- Em resumo: Comportamento das funções de custo para valores grandes de n .

DETALHES DE NOTAÇÃO

- Inicialmente utilizamos $f(n)$ para denotar uma função de complexidade obtida através da análise de um algoritmo.
- Ex: $f(n) = n^3 + n + \log(n)$.

DETALHES DE NOTAÇÃO

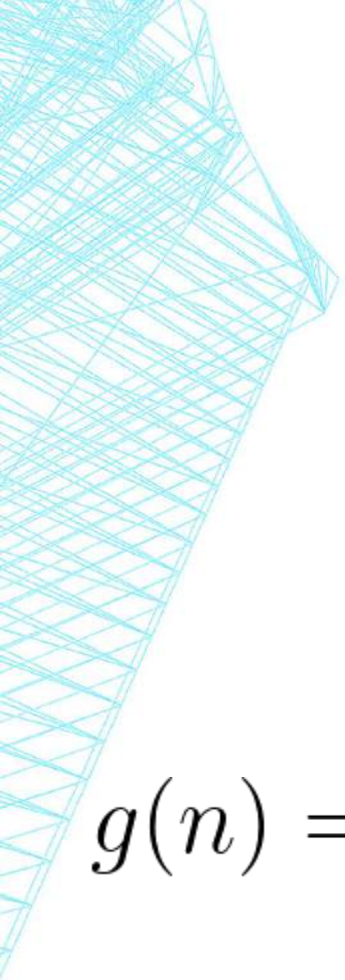
- Agora irá ocorrer uma pequena mudança na notação, a função de complexidade obtida da análise de um algoritmo (ou fornecida) será denotada por $g(n)$.
- $f(n)$ irá denotar uma segunda função de complexidade que queremos descobrir se domina assintoticamente $g(n)$.

NOTAÇÃO O

- A função $f(n)$ domina assintoticamente a função $g(n)$ se existem duas constantes positivas c e m tais que, para $n \geq m$, tem-se:
 - $|g(n)| \leq c \times |f(n)|$
- Notação: $g(n) = O(f(n))$

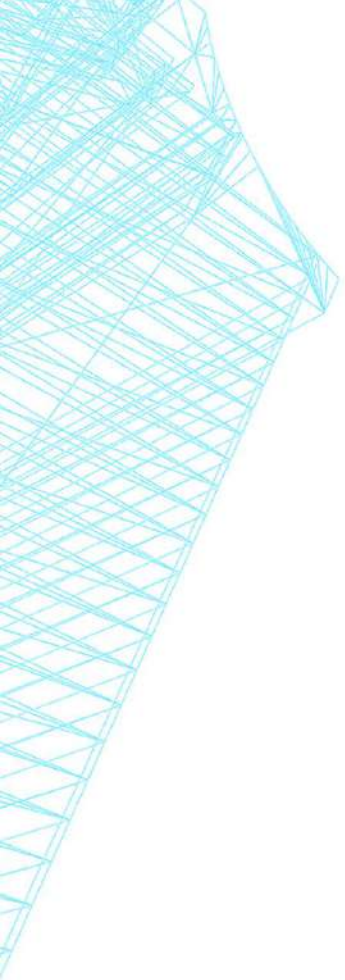
NOTAÇÃO O

- $g(n) = O(f(n))$
 - existe uma constante positiva c , tal que $g(n) \leq c * f(n)$, para algum $n \geq n_0$.
 - n_0 é o ponto de referência no gráfico em $c * f(n)$ passa a ser, sempre, maior que $g(n)$.



NOTAÇÃO O

$$g(n) = O(f(n)), \exists c > 0 \text{ e } m \mid 0 \leq g(n) \leq cf(n), \forall n \geq m$$



NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2		
4		
8		
16		
32		
64		

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

Para $c = 4$ e $n_0 = 16$, $g(n) = O(f(n)) = O(n^2)$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = 4$$

n	4 x f(n)	g(n)
2	16	32
4	64	88
8	256	272
16	1024	928
32	4096	3392
64	16384	12928

Para $c = 4$ e $n_0 = 16$, $g(n) = O(f(n)) = O(n^2)$

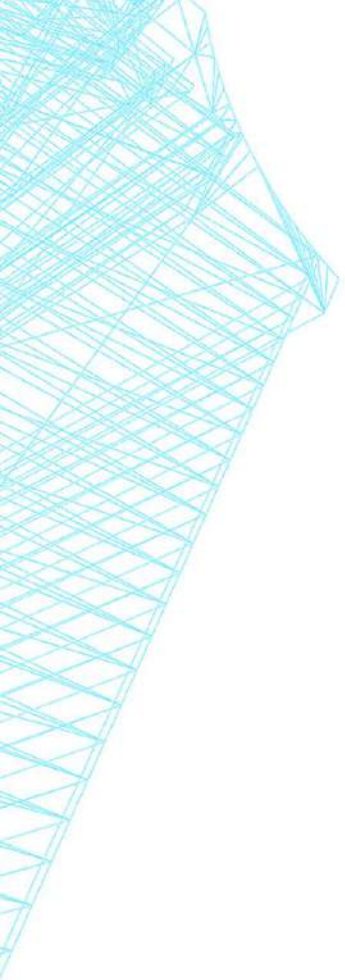
NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$n^2 + 10n \leq c.n^2$$



DEFINIÇÃO DE LIMITE

“O limite descreve o comportamento de uma função à medida que as entradas ficam cada vez mais próximas de um valor específico, sem necessariamente atingir esse valor.”

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$n^2 + 10n \leq c.n^2 \quad \div (n^2) \text{ ambos os lados}$$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$3n^2 + 10n \leq c \cdot n^2 \div (n^2) \text{ ambos os lados}$$

$$3 + \frac{10}{n} \leq c$$

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$3n^2 + 10n \leq c \cdot n^2 \div (n^2) \text{ ambos os lados}$$

$$3 + \frac{10}{n} \leq c$$

Quando $n \rightarrow \infty$, $\frac{10}{n}$ tende a 0, mas **nunca será 0**, logo C pode assumir qualquer valor **acima de 4** (pode demorar a encontrar o n_0)

NOTAÇÃO O

$$f(n) = n^2$$

$$g(n) = 3n^2 + 10n$$

$$C = ?$$

$$3n^2 + 10n \leq c \cdot n^2 \div (n^2) \text{ ambos os lados}$$

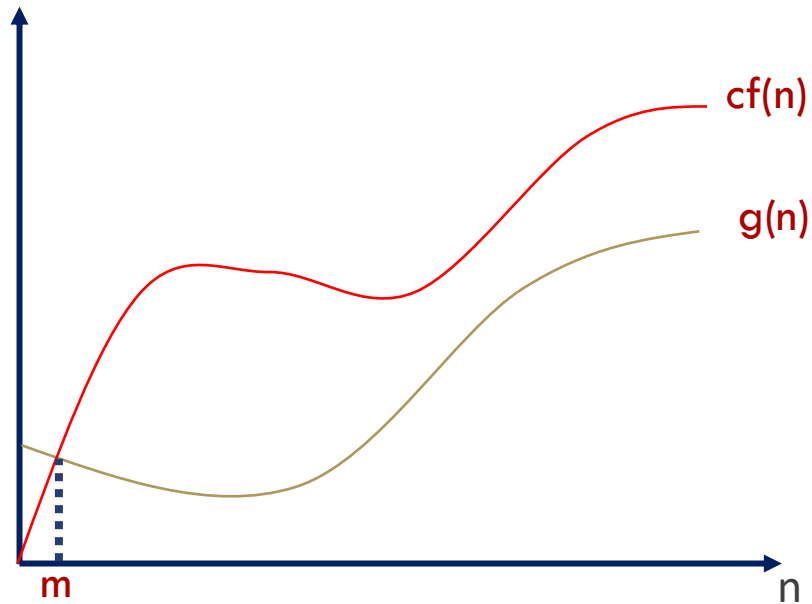
$$3 + \frac{10}{n} \leq c$$

Para facilitar. Para $n_0=1$, teremos que $c \geq 13$ (leva a uma prova mais rápida)

NOTAÇÃO O

$$g(n) = O(f(n))$$

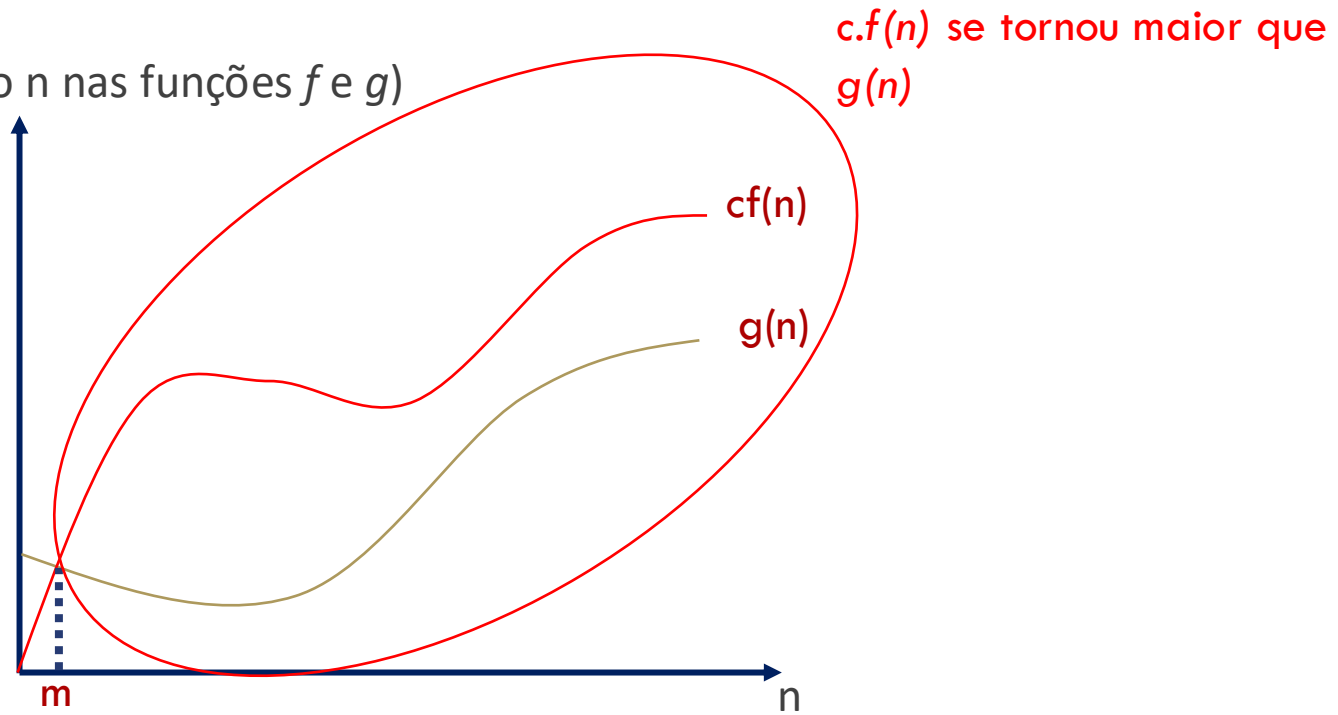
Custo (substituindo n nas funções f e g)



NOTAÇÃO O

$$g(n) = O(f(n))$$

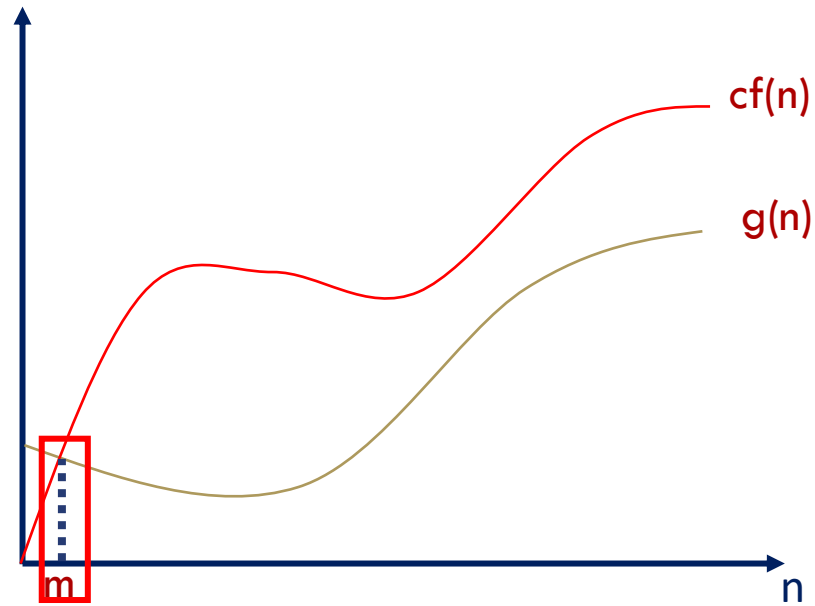
Custo (substituindo n nas funções f e g)



NOTAÇÃO O

$$g(n) = O(f(n))$$

Custo (substituindo n nas funções f e g)



A partir de um certo valor de n (denominado m)

NOTAÇÃO O

- $g(n) = O(f(n))$ implica que $f(n)$ é um limite superior para a taxa de crescimento de $g(n)$.
- Ex: Se o tempo de execução $S(n)$ de um algoritmo é $O(n^2)$, existem constantes c e m tais que, para valores de $n \geq m$, $T(n) \leq c * n^2$.
- $S(n) = O(n^2)$: dizemos que $S(n)$ é da ordem de n^2 .

NOTAÇÃO O

- Se um algoritmo é $O(f(n))$, ele também será $O(g(n))$ para toda função $g(n)$, tal que $g(n) > f(n)$.
- Se um algoritmo é $O(n^2)$ ele também é $O(n^3)$, $O(n^3)$, $O(2^n)$, ...
 - Limite assintoticamente justo.

PROPRIEDADES / OPERAÇÕES

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \quad c = \text{constante}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

$$O(O(f(n))) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

$$f(n)O(g(n)) = O(f(n)g(n))$$

PROPRIEDADES / OPERAÇÕES

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \quad c = \text{constante}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

$$O(O(f(n))) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

$$f(n)O(g(n)) = O(f(n)g(n))$$

- Algoritmo com três trechos cujos tempos são $O(n)$, $O(n^2)$ e $O(n)$:

- *Qual será a complexidade?*

PROPRIEDADES / OPERAÇÕES

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \quad c = \text{constante}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

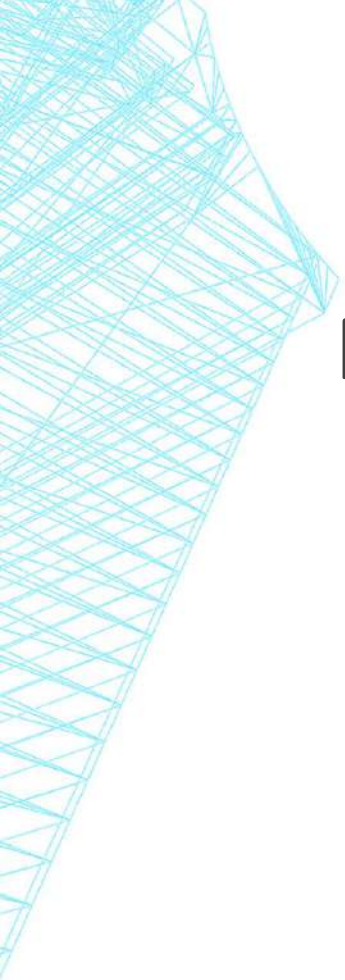
$$O(O(f(n))) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

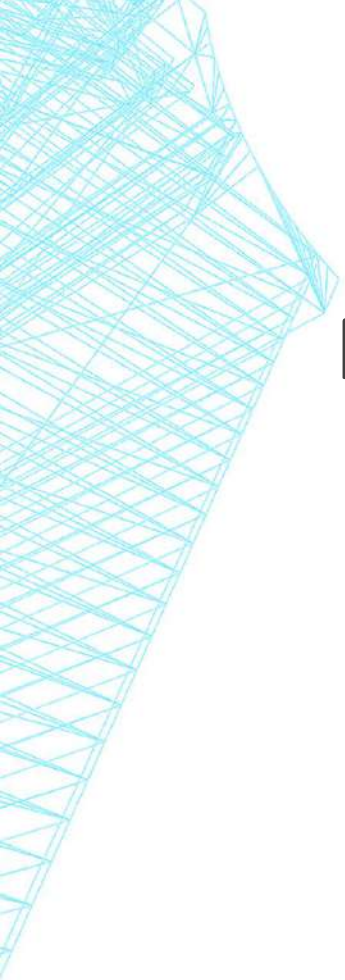
$$f(n)O(g(n)) = O(f(n)g(n))$$

- Algoritmo com três trechos cujos tempos são $O(n)$, $O(n^2)$ e $O(n)$:
 - O algoritmo é $O(n^2)$.



EXERCÍCIO

Prove que $3n^3 + 2n^2 + n + 1 = O(n^3)$



EXERCÍCIO

Prove que $n^{n+1} = O(2^n)$

EXERCÍCIO

Prove que $n^{n+1} = O(2^n)$

$$n^{n+1} \leq c 2^n$$

$$\frac{n \cdot n^n}{2^n} \leq c \quad \text{Divide os dois lados por } 2^n$$

$$n \cdot \left(\frac{n}{2}\right)^n \leq c$$

EXERCÍCIO

Prove que $n^{n+1} = O(2^n)$

$$n^{n+1} \leq c 2^n$$

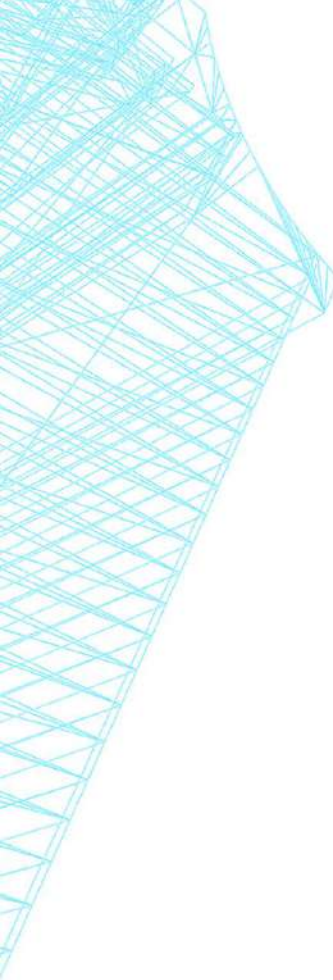
$$\frac{n \cdot n^n}{2^n} \leq c$$

$$n \cdot \left(\frac{n}{2}\right)^n \leq c$$

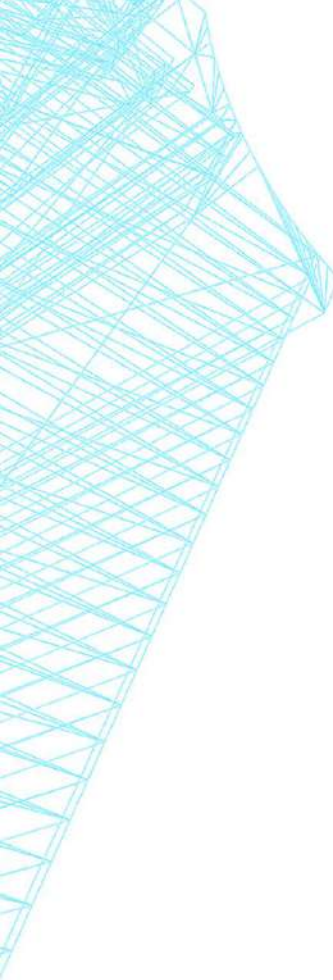
Vamos analisar o comportamento do lado esquerdo da inequação à medida que n cresce:

- Para qualquer $n > 2$, a fração $(n/2)$ é estritamente maior que 1.
- Consequentemente, o termo $(n/2)^n$ cresce exponencialmente.
- Multiplicar esse termo por n faz com que a expressão cresça ainda mais rápido.

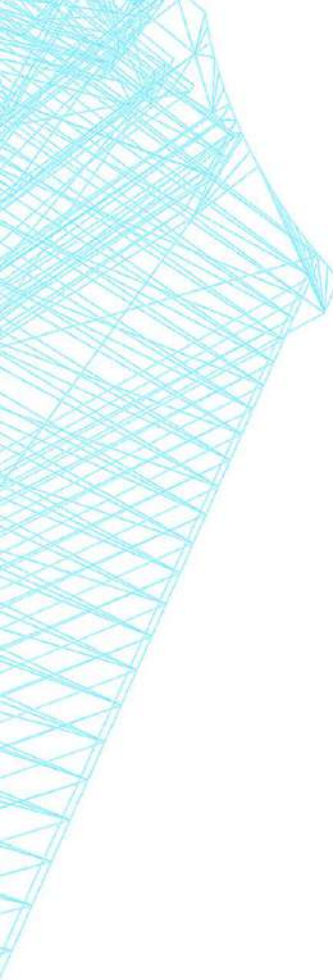




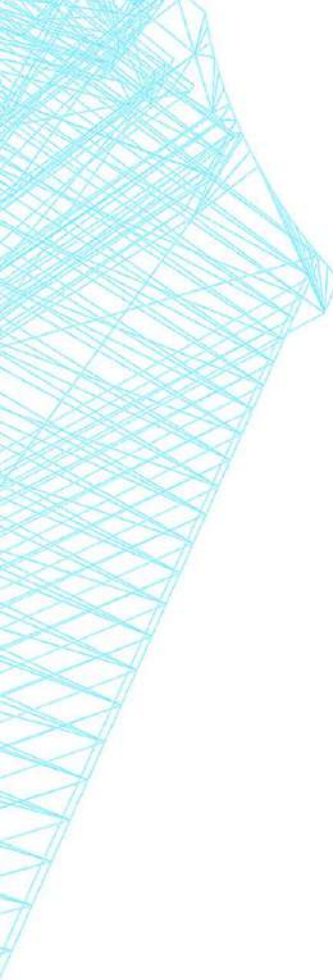
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor]) ← O(1)
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```



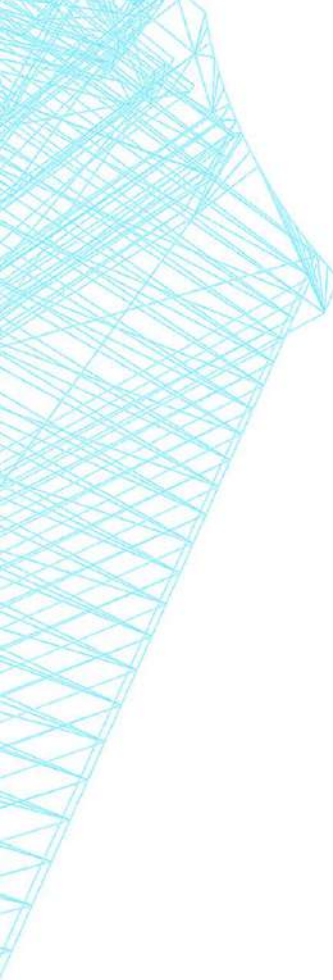
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor]) ← O(1)
7.                 menor = j; ← O(1)
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} $O(1) + O(1) = O(1)$


```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ (n-1)+(n-2)+(n-3)...
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} $O(1) + O(1) = O(1)$



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n-1)+O(n-2)+O(n-3)...
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} $O(1) + O(1) = O(1)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n-1)
6.             if (array[j] < array[menor])
7.                 menor = j;           } O(1) + O(1) = O(1)
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){  $O(n-1) = O(n)$ 
6.             if (array[j] < array[menor])
7.                 menor = j;           }  $O(1) + O(1) = O(1)$ 
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

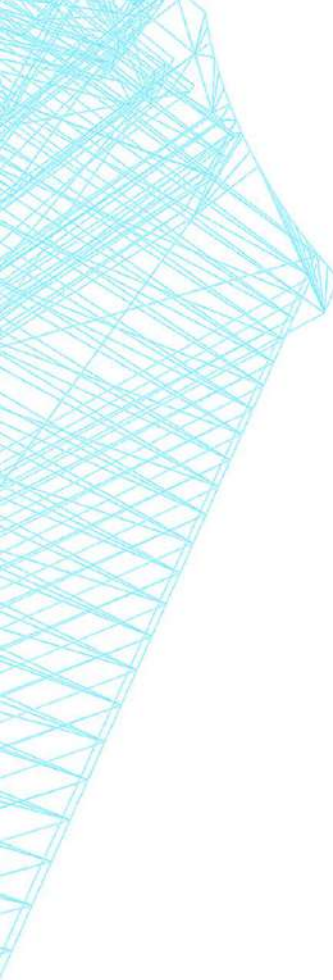
```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n)
6.             if (array[j] < array[menor])
7.                 menor = j;           } O(1)
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){ O(n)
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(1)$ $O(n)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$

$O(1)$

$O(1)$

$O(1)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$

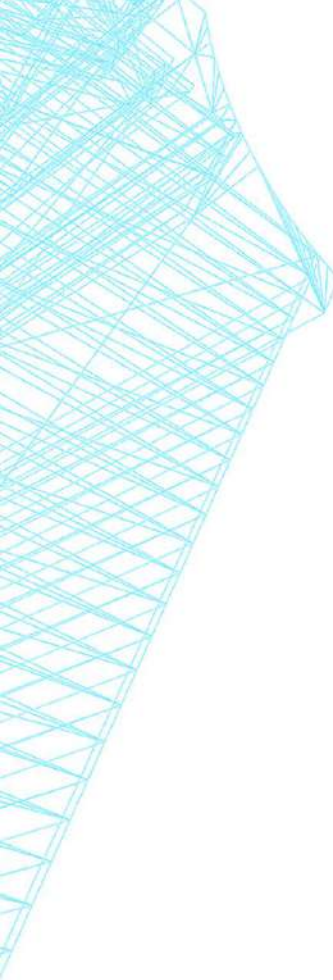
$O(1)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n)$

$O(1)$

$O(n)+O(1) = O(n)$

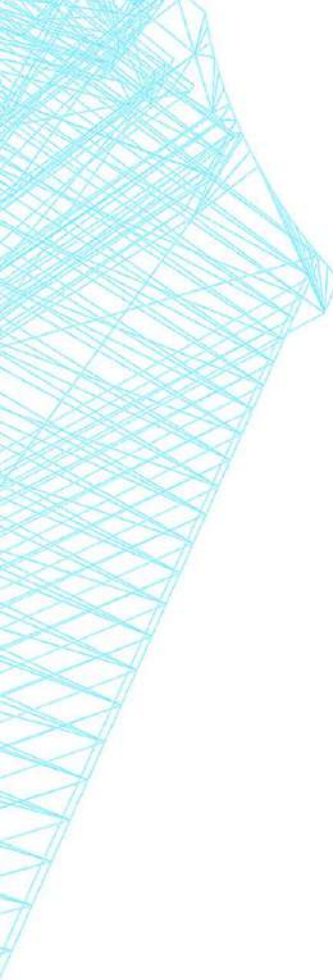


```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} $O(n)$

```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){ O(n-1) = O(n)
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

} O(n)



```
1. void algZ(int []array){
2.     int tam = array.lenght;
3.     for(int pos = 0; pos<tam-1; pos++){
4.         int menor = pos;
5.         for(int j = pos+1; j<tam; j++){
6.             if (array[j] < array[menor])
7.                 menor = j;
8.         }
9.         int aux = array[menor];
10.        array[menor] = array[pos];
11.        array[pos] = aux;
12.    }
13.}
```

$O(n^2)$

NOTAÇÃO Ω

- **Limite assintótico inferior** do comportamento de um algoritmo.
- Se uma função é $\Omega(n^2)$;
- ela também será limitada assintoticamente por funções de graus inferiores.

NOTAÇÃO Ω

- Uma função $g(n)$ é $\Omega(f(n))$ se existirem duas constantes c e m tais que $g(n) \geq c * f(n)$, para todo $n \geq n_0$.

NOTAÇÃO Ω

- Uma função $g(n)$ é $\Omega(f(n))$ se existirem duas constantes c e m tais que $g(n) \geq c * f(n)$, para todo $n \geq m$.

NOTAÇÃO Ω – NOTAÇÃO MATEMÁTICA

$$g(n) = \Omega(f(n)), \exists c > 0 \text{ e } m \mid 0 \leq cf(n) \leq g(n), \forall n \geq m$$

NOTAÇÃO Ω

$$f(n) = 5n^2 + 10$$
$$g(n) = n^3 + 2n^2$$

n	g(n)	f(n)
0		
2		
4		
6		
8		

NOTAÇÃO Ω

$$f(n) = 5n^2 + 10$$
$$g(n) = n^3 + 2n^2$$

n	g(n)	f(n)
0	0	10
2	16	30
4	96	90
6	288	190
8	640	330

NOTAÇÃO Ω

$$f(n) = 5n^2 + 10$$
$$g(n) = n^3 + 2n^2$$

n	g(n)	f(n)
0	0	10
2	16	30
→ 4	96	90
→ 6	288	190
→ 8	640	330

NOTAÇÃO Ω

$$f(n) = 5n^2 + 10$$
$$g(n) = n^3 + 2n^2$$

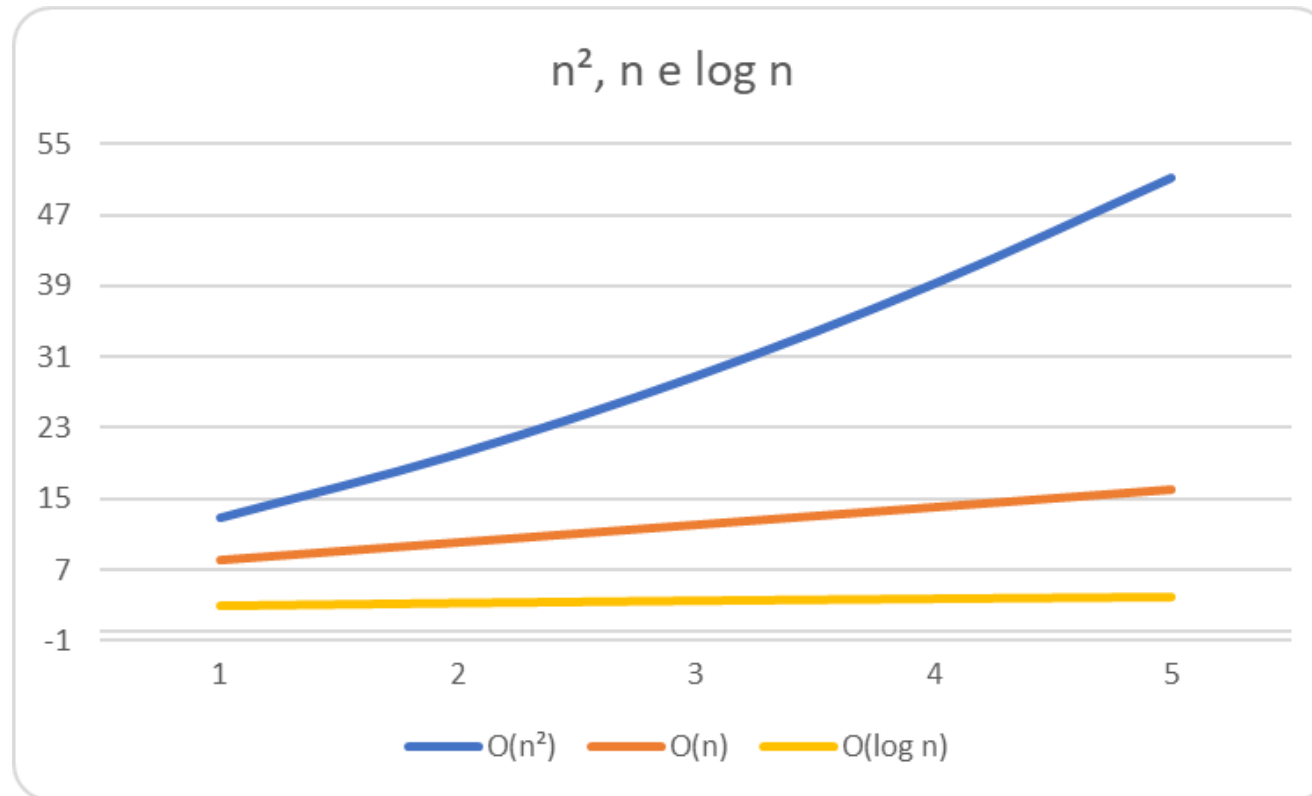
n	g(n)	f(n)
0	0	10
2	16	30
→ 4	96	90
→ 6	288	190
→ 8	640	330
?	?	

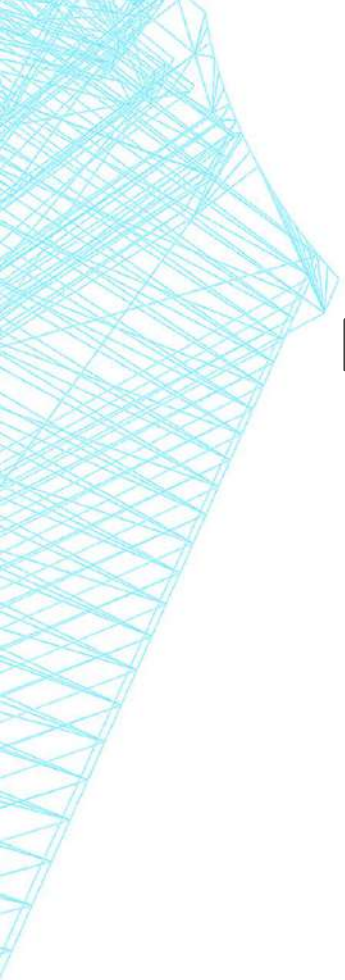
Para $c = 1$ e $m = 4$, $g(n) = (f(n)) = (5n^2 + 10)$

NOTAÇÃO Ω

- $g(n) = \Omega(f(n))$ implica que $f(n)$ é um limite inferior para o crescimento de $g(n)$.

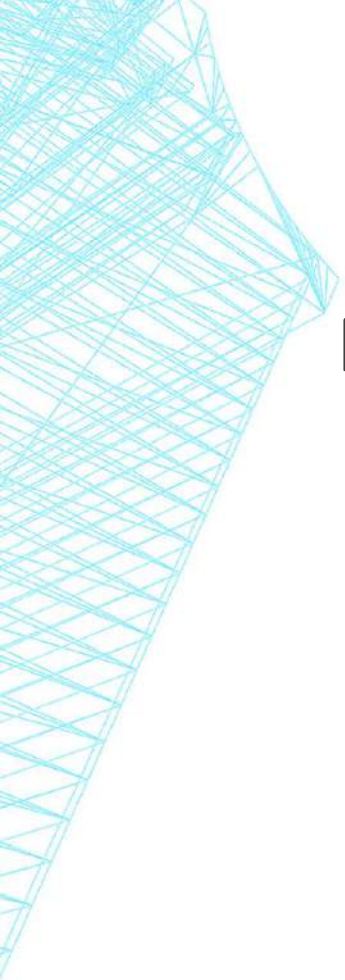
LIMITES SUPERIOR E INFERIOR





EXERCÍCIO

Prove que $n^2 + 10 = \Omega(n^2)$



EXERCÍCIO

Prove que $3n^3 + 2n^2 = \Omega(n^3)$

NOTAÇÃO Θ

- $g(n) = \theta(f(n))$ implica que $f(n)$ é um limite assintótico justo para o crescimento de $g(n)$.

NOTAÇÃO Θ

- Uma função $g(n)$ é $\Theta(f(n))$ se existirem constantes $c1$, $c2$ e m tais que

- $0 \leq c1 * f(n) \leq g(n) \leq c2 * f(n)$

- para todo $n \geq n_0$.

NOTAÇÃO Θ

- Uma função $g(n)$ é $\Theta(f(n))$ se existirem constantes $c1$, $c2$ e m tais que

- $0 \leq c1 * f(n) \leq g(n) \leq c2 * f(n)$

- para todo $n \geq m$.

NOTAÇÃO Θ – DEFINIÇÃO MATEMÁTICA

$$g(n) = \Theta(f(n)), \exists c_1 > 0, c_2 > 0 \text{ e } m \mid 0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq m$$

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

$$c_2 = 5$$

n	1*f(n)	g(n)	5*f(n)
0			
2			
4			
6			
8			

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

$$c_2 = 5$$

n	1*f(n)	g(n)	5*f(n)
0	0	10	0
2	8	30	40
4	24	90	120
6	48	190	240
8	80	330	400

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

$$c_2 = 5$$

n	1*f(n)	g(n)	5*f(n)
0	0	10	0
2	8	30	40
4	24	90	120
6	48	190	240
8	80	330	400

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

$$c_2 = 5$$

n	1*f(n)	g(n)	5*f(n)
0	0	10	0
2	8	30	40
4	24	90	120
6	48	190	240
8	80	330	400

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

$$c_2 = 5$$

n	1*f(n)	g(n)	5*f(n)
0	0	10	0
2	8	30	40
4	24	90	120
6	48	190	240
8	80	330	400

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

$$c_2 = 5$$

n	1*f(n)	g(n)	5*f(n)
0	0	10	0
2	8	30	40
4	24	90	120
6	48	190	240
8	80	330	400

NOTAÇÃO Θ

$$f(n) = n^2 + 2n$$

$$g(n) = 5n^2 + 10$$

$$c_1 = 1$$

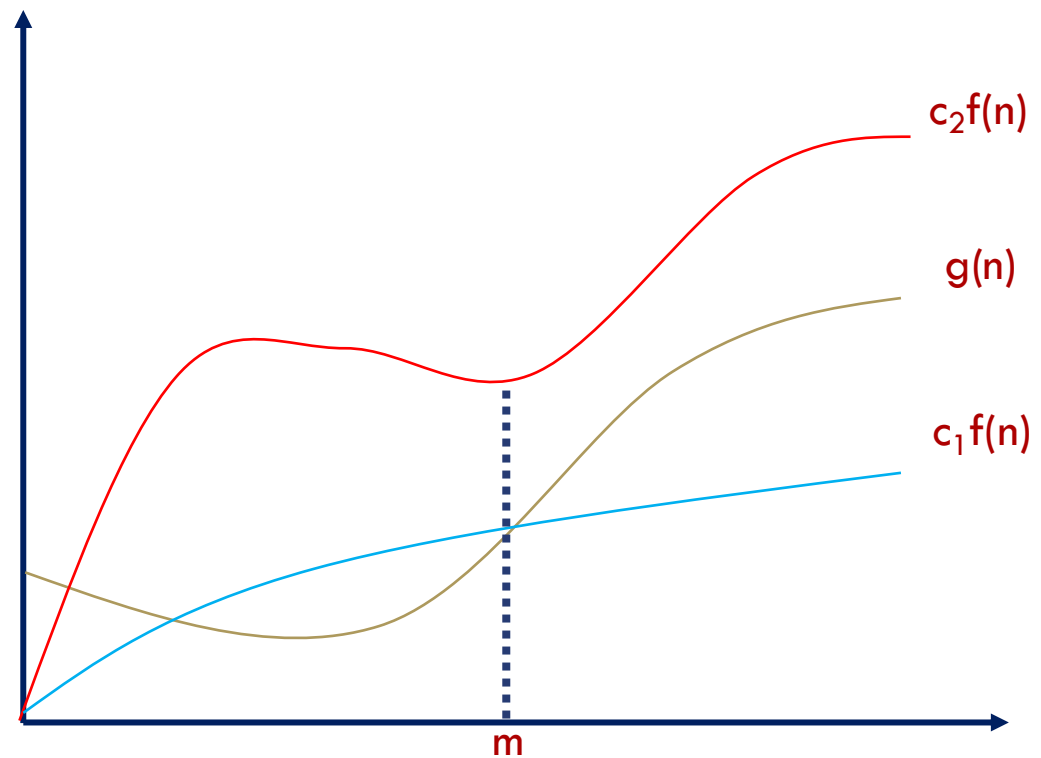
$$c_2 = 5$$

$$0 \leq 1 * f(n) \leq g(n) \leq 5 * f(n), \text{ para } n_0 = 2$$

n	1*f(n)	g(n)	5*f(n)
0	0	10	0
2	8	30	40
4	24	90	120
6	48	190	240
8	80	330	400

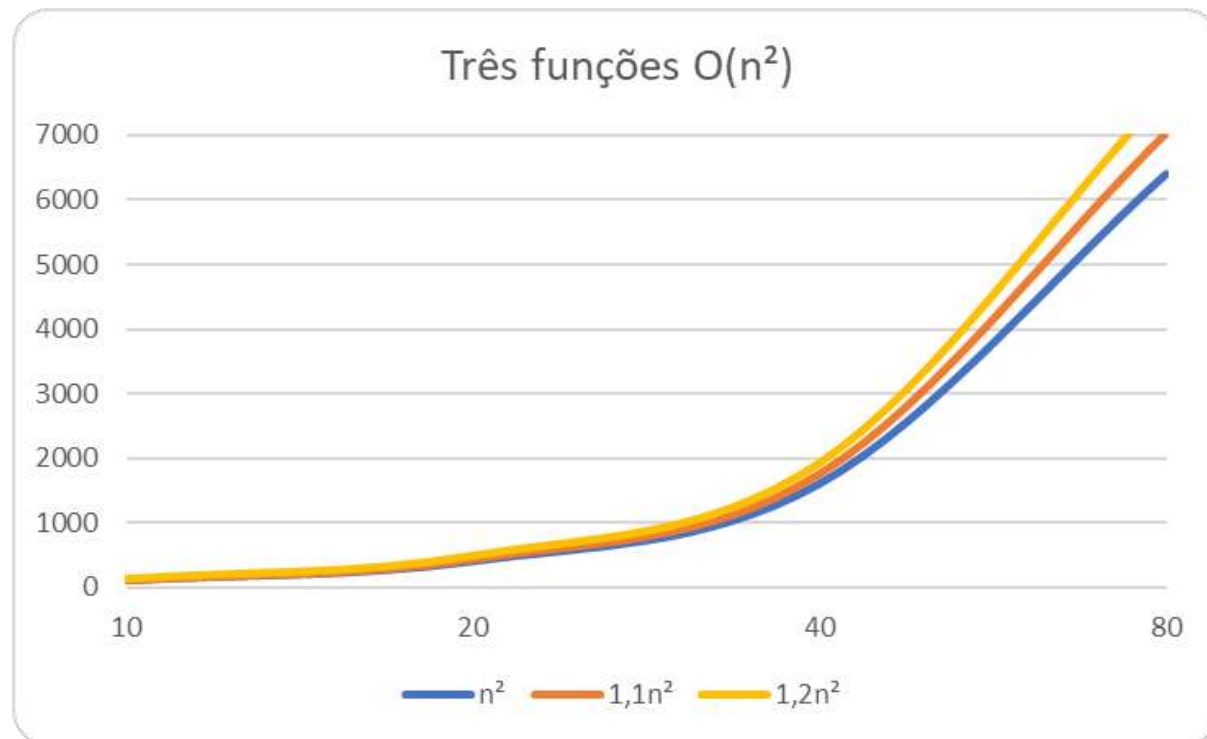
NOTAÇÃO Θ

$$g(n) \text{ é } \theta(f(n))$$



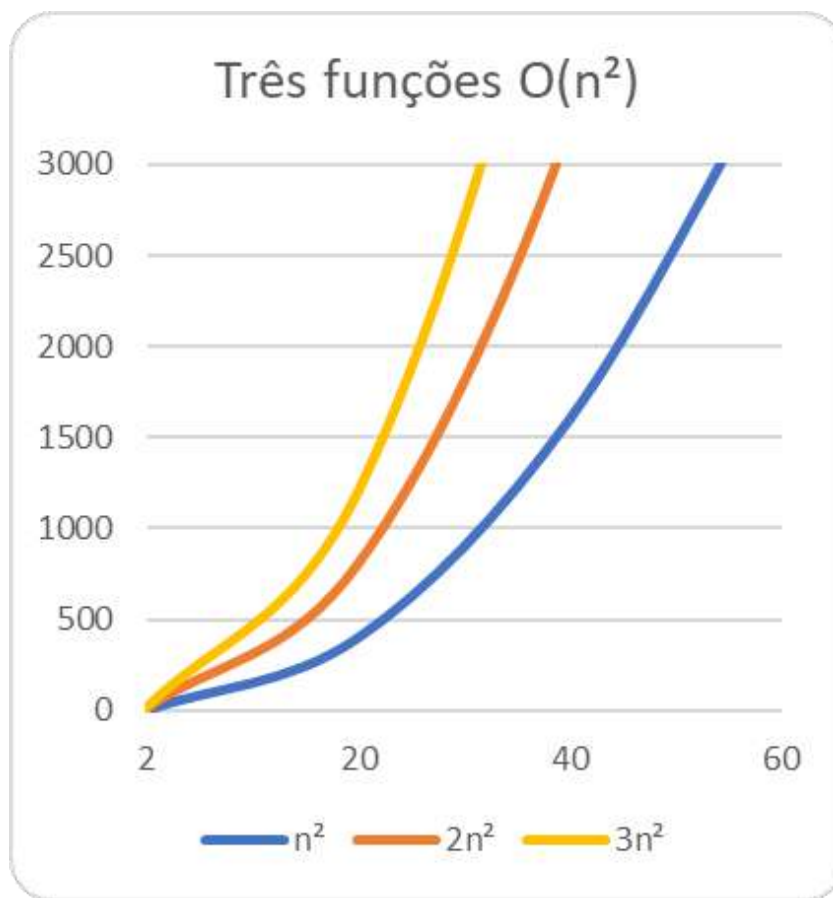
NOTAÇÃO Θ

- $g(n) = \theta(f(n))$ implica que temos, em $f(n)$, um limite assintótico firme/justo para $g(n)$.



NOTAÇÃO Θ

- $g(n) = \theta(f(n))$ implica que temos, em $f(n)$, um limite assintótico firme/justo para $g(n)$.





CLASSES DE COMPORTAMENTOS

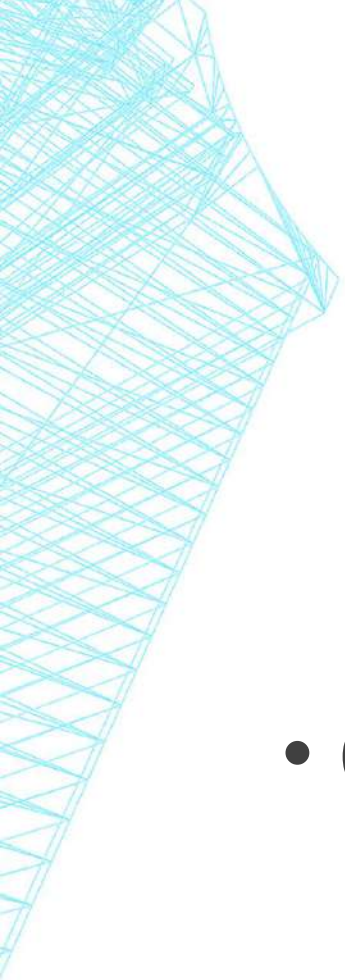
CLASSES DE COMPORTAMENTO

- Seja $f(n)$ a função de complexidade de um algoritmo.
- Então, dizemos que $O(f(n))$ é complexidade assintótica do algoritmo.
- Problemas com comportamento similar podem ter a mesma complexidade assintótica:
 - Classes de comportamento assintótico



CLASSES DE COMPORTAMENTO

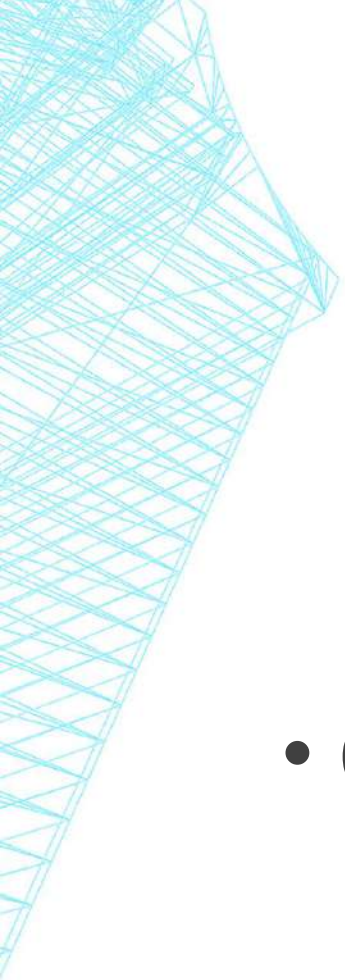
- Algoritmos da mesma classe de comportamento apresentam características típicas.
 - Porém, atenção para as exceções.



COMPLEXIDADE FIXA OU CONSTANTE

$$f(n) = O(1)$$

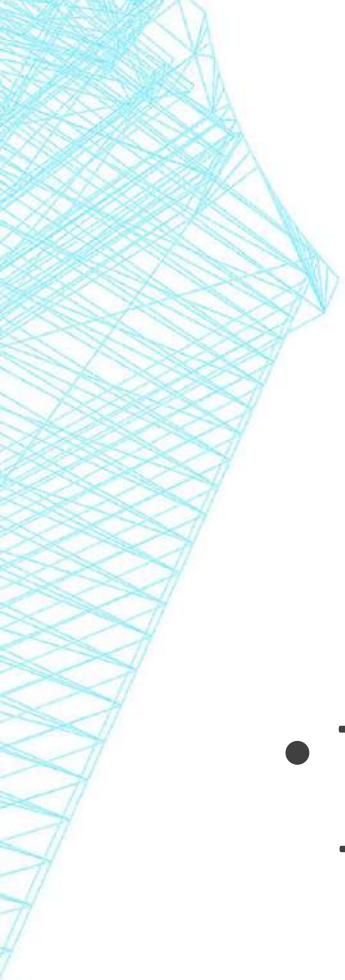
- O custo do algoritmo independe do tamanho de n .
- As instruções do algoritmo são executadas um número fixo de vezes.



COMPLEXIDADE FIXA OU CONSTANTE

$$f(n) = O(c)$$

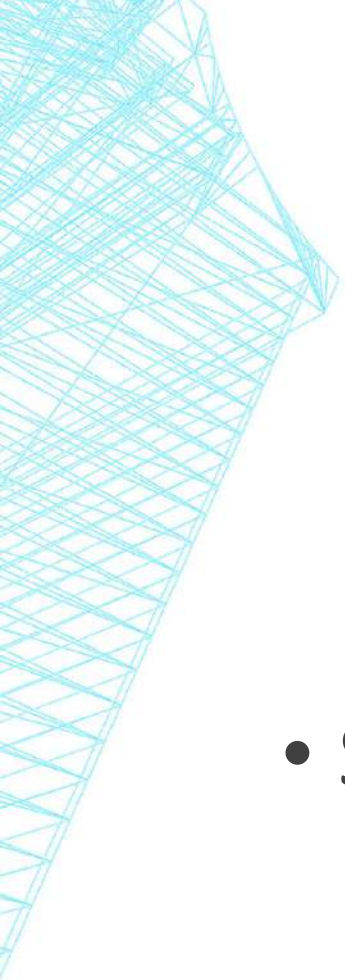
- O custo do algoritmo independe do tamanho de n .
- As instruções do algoritmo são executadas um número fixo de vezes.



COMPLEXIDADE LOGARÍTMICA

$$f(n) = O(\log n)$$

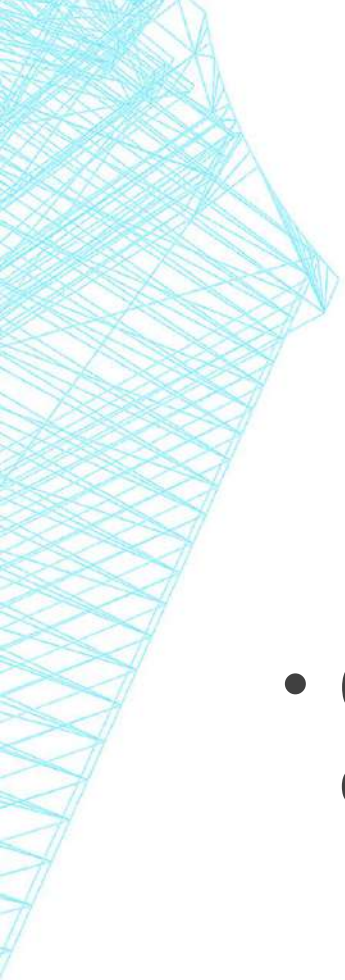
- Tipicamente algoritmos que resolvem problemas transformando-os em problemas menores.



COMPLEXIDADE LOGARÍTMICA

$$f(n) = O(\log n)$$

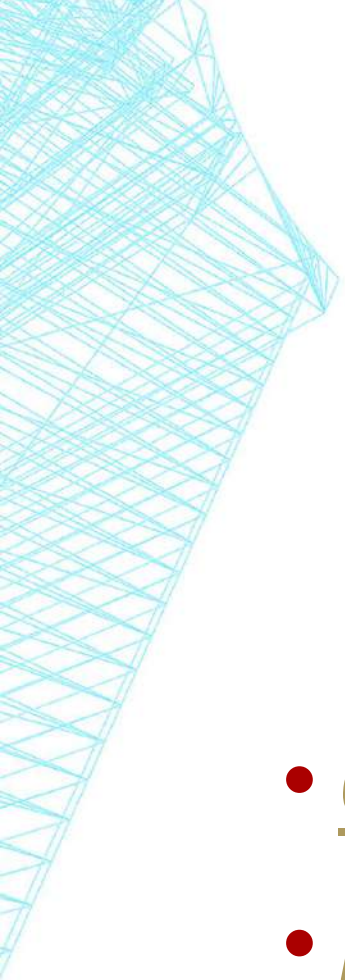
- Supondo que a base do logaritmo seja 2:
 - Para $n = 1.000$, $\log_2 \cong 10$
 - Para $n = 1.000.000$, $\log_2 \cong 20$



COMPLEXIDADE LINEAR

$$f(n) = O(n)$$

- Operações realizadas sobre cada elemento dos dados de entrada.
 - Ex: operações lineares em vetores.
- Cada vez que n dobra de tamanho, o tempo de execução também dobra.



ALGORITMOS $N \times \log N$

$$f(n) = O(n \log n)$$

- Quebram o problema em partes menores;
- Resolvem cada um deles independentemente;
- Agrupam as soluções.

ALGORITMOS $N \times \log N$

$$f(n) = O(n \log n)$$

- Supondo que a base do logaritmo seja 2:
 - Para $n = 1.000.000$, $n \log_2 n \cong 20.000.000$
 - Para $n = 2.000.000$, $n \log_2 n \cong 42.000.000$

ALGORITMOS QUADRÁTICOS

$$(n) = O(n^2)$$

▪ Ex: **n**

n²

10

100

100

10.000

1.000

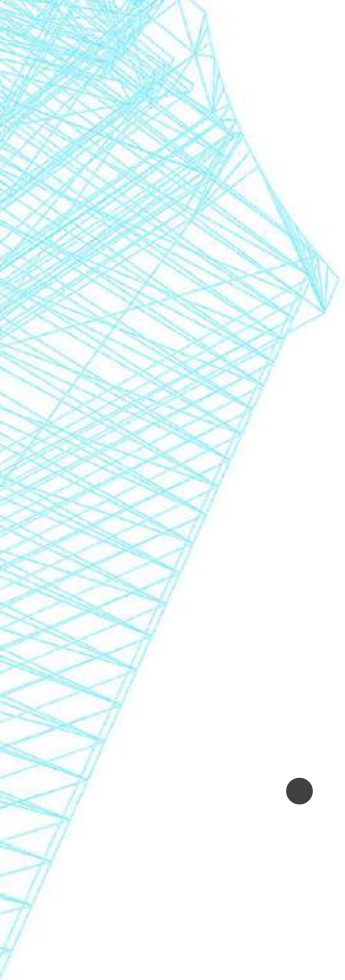
1.000.000

10.000

100.000.000

100.000

10.000.000.000



ALGORITMOS CÚBICOS

$$f(n) = O(n^3)$$

- Úteis apenas para resolver problemas relativamente pequenos.
 - Sempre que n dobra, o tempo de execução é multiplicado por 8.

ALGORITMOS CÚBICOS

$$f(n) = O(n^3)$$

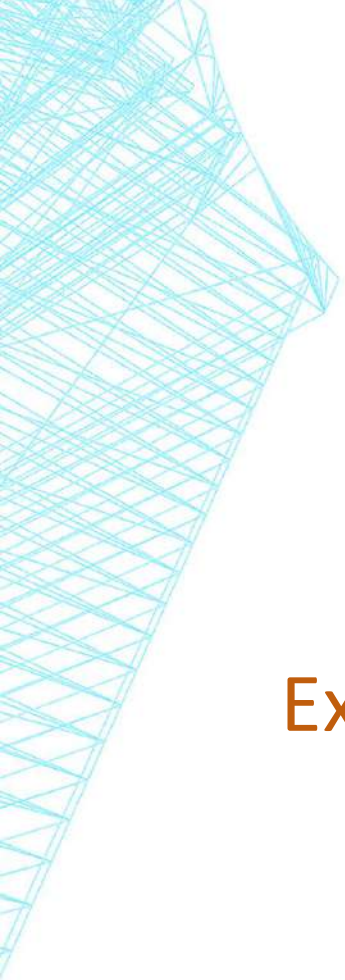
Ex:

n	n^3
10	1.000
100	1.000.000
1.000	1.000.000.000
10.000	1.000.000.000.000
100.000	100.000.000.000.000

ALGORITMOS EXPONENCIAIS

$$f(n) = O(2^n) \text{ ou } f(n) = O(c^n)$$

- Não são úteis sob o ponto de vista prático.
- Em geral, ocorrem na solução de problemas por algoritmos utilizando força bruta.
 - Sempre que n dobra, o tempo de execução fica elevado ao proporcionalmente a c .

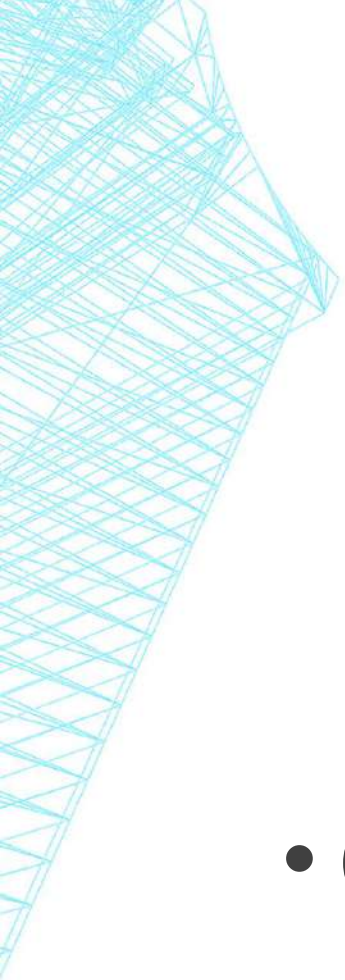


ALGORITMOS EXPONENCIAIS

$$f(n) = O(2^n)$$

Ex:

n	2^n
10	1.024
20	1.048.576
40	1.099.511.627.776
80	1.208.925.819.614.629.174.706.176



ALGORITMOS FATORIAIS

$$f(n) = O(n!)$$

- Comportamento ainda pior do que $O(2^n)$
 - Multiplicação crescente: $x2, x3, x4, x5 \dots$

ALGORITMOS FATORIAIS

$$f(n) = O(n!)$$

Ex:	n	n!
	10	3.628.800
	15	1.307.674.368.000
	20	2.432.902.008.176.640.000
	25	15.511.210.043.330.985.984.000.000

$$f(n) = O(n!)$$

- Ex: Para $n = 40$, $40! \cong$
816.000.000.000.000.000.000.000.000.000.000.000.0
00.000.000.000
- um número com **48 dígitos!**

ALGORITMOS FATORIAIS

$$f(n) = O(n!)$$

- $40! \approx 8,16 \times 10^{47}$
- Frontier: $1,685 \times 10^{18}$ instruções/segundo.
- $\frac{8,16 \times 10^{47}}{1,685 \times 10^{18}} = 4,84 \times 10^{29}$ segundos.
- = 15.356.195.999.259.289.369 milênios.

TEMPOS DE EXECUÇÃO

Um computador que execute
1 bilhão (10^9) de instruções por segundo

	10	20	30	40	50	60
n	0,00000001	0,00000002	0,00000003	0,00000004	0,00000005	0,00000006
n ²	0,0000001	0,0000004	0,0000009	0,0000016	0,0000025	0,0000036
n ³	0,000001	0,000008	0,000027	0,000064	0,000125	0,000216
n ⁵	0,0001	0,0032	0,0243	0,1024	0,3125	0,7776
2 ⁿ	0,000001024	0,001048	1,0737418	1.099,5 s = 18 min	1.125.899 s = 312 h	1.152.921.504 s = 36,5 anos
3 ⁿ	0,000059049	3,4867 s	205.891 s = 57,2 h	12.157.665.459 s = 385 anos	227.488 séculos	134 milhões de séculos